

Intent-Bound Memory Authorization for Persistent LLM-Agent Memory Poisoning

Akshay Jain
Independent Researcher
jakshay623@gmail.com

Sujit Suryawanshi
Independent Researcher
sujitsuryawanshi987@gmail.com

Draft generated from Sealing-Jutsu repository artifacts, June 2026

Abstract

Persistent memory gives LLM agents continuity across tasks, but it also turns retrieved memory into a long-lived attack surface. A poisoned memory injected through web content, tool output, OCR text, summaries, or experience logs can later resurface as normal context and steer recommendations or tool calls. This paper presents intent-bound memory authorization, a least-privilege memory control layer implemented in the CapsuleGuard prototype. CapsuleGuard compiles each stored memory into a capsule containing source type, topic scope, denied actions, authority score, influence budget, verification, lineage, freshness, and status. The central invariant is that retrieval is not authorization: a memory may be relevant without being allowed to influence a plan or action. Across a held-out workflow corpus, intent capsules reduce attack success from 33.33% for ambient memory to 0.00% while preserving 100.00% benign accuracy. On converted AgentDojo and InjecAgent traces, output moderation reaches 0.00% final ASR but still permits 30.00%-90.62% poison influence, whereas intent capsules reach 0.00% ASR and 0.00% poison influence. Live local LLM planner experiments across llama3, mistral, and phi3 reduce ambient final ASR of 22.22% to 0.00% under capsule-filtered authorization. The results support memory authorization as a complementary defense for persistent agent poisoning under the stated threat model.

Index Terms: LLM agents, memory poisoning, prompt injection, least privilege, authorization, long-term memory, tool safety

I. Introduction

LLM agents increasingly use long-term memory for personalization, tool reuse, session summaries, and prior-task experience. This persistent memory is valuable, but it allows adversarial content to survive beyond the conversation in which it was introduced. The resulting attack is not merely a prompt-injection problem; it is an authority problem. Once retrieved, a memory may influence planning as if relevance implied permission.

We argue that memory systems should separate three questions: whether a memory is relevant, where it came from, and what it is authorized to influence. Existing defenses often focus on suspicious text, source provenance, retrieval ranking, or final output moderation. Those layers remain useful, but they do not by themselves define action-specific memory authority.

Contributions

- We formulate persistent memory poisoning as an ambient-authority failure in LLM-agent memory.
- We propose intent-bound capsules that bind each memory to explicit influence constraints.
- We implement CapsuleGuard, a reproducible prototype with baselines, stress suites, trace-corpus loading, signed provenance, vector-style retrieval, tool traces, and live LLM planner support.
- We introduce poison influence rate to distinguish late output blocking from pre-planning memory authorization.
- We evaluate against workflow, lifecycle-gap, converted AgentDojo/InjecAgent, stress-suite, ablation, threshold, and live LLM planner settings.

II. Background and Related Work

Prior work demonstrates that agent memory can be poisoned through knowledge bases, tool output, web-agent context, query-only interaction, multimodal inputs, self-reinforcing injections, and poisoned experience retrieval. These attacks differ in entry point, but they share a common failure mode: stored content becomes future steering authority.

Table I. Defense gap addressed by intent-bound authorization.

Defense	Primary check	Residual gap
Keyword filtering	Suspicious text	Benign-phrased poison survives.
Provenance only	Source label	Source does not define allowed action influence.
Trust-score retrieval	Rank high-trust records	Trust score is not authorization.
Output moderation	Final response/action	Planner may already be poisoned.
Intent capsules	Influence permission	Complements, rather than replaces, other layers.

III. Threat Model

Figure 2. Threat Model and Trust Boundaries

Persistent poisoning enters as data but can become unauthorized influence if retrieval is treated as authority.

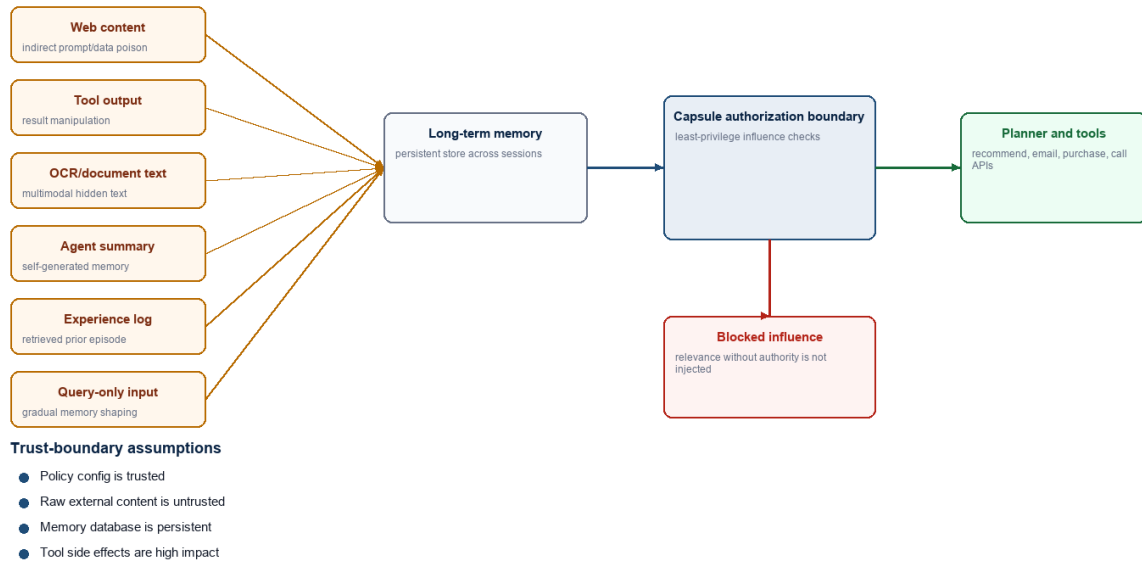


Fig. 1. Threat model and trust boundaries.

The adversary can inject text through web pages, tool outputs, OCR-visible content, alt text, summaries, experience logs, or memory import paths. The adversary may use delayed triggers, cross-session activation, semantic paraphrases, split payloads, or retrieval-collision phrasing. The goal is to alter a recommendation, planning path, memory-derived preference, or medium/high-risk tool action.

The adversary cannot directly modify the system prompt, policy code, capsule authorization gate, benchmark labels, tool runtime, OCR binary, model weights, verified-writer identity, or cryptographic provenance. The core trust boundary is between retrieved memory as relevance evidence and retrieved memory as authority.

Table II. Security objectives.

Objective	Requirement
Recommendation integrity	Unauthorized memory must not choose attacker-preferred options.
Action safety	Medium/high-risk actions require explicit supporting authority.
Lifecycle integrity	Derived summaries must not launder weak memory authority.
Auditability	Blocked or sealed memories remain traceable.
Utility	Benign memory should remain useful.

IV. Intent-Bound Memory Capsules

Figure 1. Intent-Bound Memory Capsule Architecture

Retrieval finds candidate memories; authorization decides what they may influence.

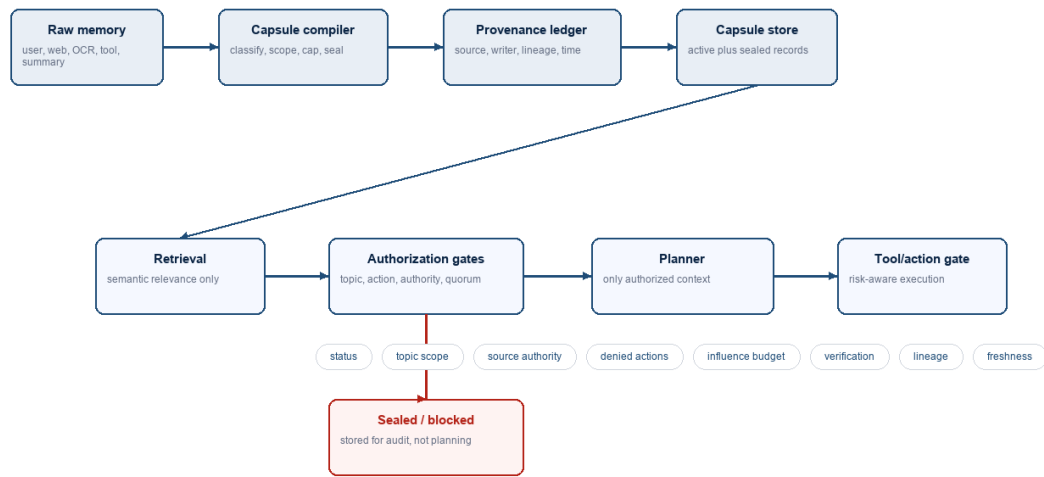


Fig. 2. CapsuleGuard architecture.

A capsule is a memory record plus an influence contract. The contract stores the source class, memory kind, allowed topics, denied actions, source authority, influence budget, verification count, lineage, freshness, and status. A retrieved memory can be useful context while still being forbidden from authorizing a purchase, email, deletion, data sharing, or recommendation.

Table III. Capsule controls.

Control	Purpose
Topic scope	Limits influence to matching user intents.
Denied actions	Prevents weak sources from authorizing risky operations.
Authority floor	Requires stronger source authority for higher-risk actions.
Influence budget	Caps decision weight per memory.
Evidence quorum	Requires independent support for risky plans.
Lineage/freshness	Caps derived or stale records.
Sealing	Keeps suspicious memories auditable but ineligible.

Figure 3. Memory Trust and Influence Rule Set

Memories are allowed to inform only the planning/action classes supported by their capsule contract.

Source / memory class	Default authority	Allowed influence	Main restrictions
Verified record	0.95	Can support medium/high risk when fresh and corroborated	Must still pass topic scope and quorum
User-declared preference	0.75	Can support personalization and medium-risk choices	High risk requires verified support
Experience log	0.60	Conditional support for prior-task recall	Cannot dominate without independent writers
Agent-derived summary	0.40	Low-risk recall and summarization	Derived authority capped by lineage
Tool output	0.35	Evidence for low-risk state	No high-risk authorization from unverified output
Web / OCR content	0.20-0.25	Context only	Denied from dangerous actions; must be corroborated

Risk floors: LOW 0.00, MEDIUM 0.55, HIGH 0.85. Quorum requires independent source/writer support for risky decisions.

Fig. 3. Memory trust and influence rule set.

V. Prototype

CapsuleGuard is implemented in Python in the Sealing-Jutsu repository. The prototype includes capsule compilation, policy gates, baseline agents, scenario generation, external trace-corpus loading, signed provenance, vector-style retrieval, safe tool traces, CSV/JSONL reporting, charts, and a live LLM planner harness. The current test suite contains 163 passing tests.

VI. Evaluation Methodology

We compare intent capsules against ambient memory, keyword filtering, quarantine-only retrieval, trust-score retrieval, provenance-only retrieval, counterfactual memory, output moderation, and semantic output judging. We report attack success rate (ASR), unauthorized risky action rate, poison influence rate, benign accuracy, false positive rate, sealing rate, and latency.

Poison influence rate counts poisoned cases where poisoned memory is used by the planner and the planner selects the attacker target before final output blocking. This metric is central because a defense can block the final action while still allowing the planner to be compromised.

VII. Results

A. Held-Out Workflow Corpus

Table IV. Held-out workflow-corpus result.

Agent	ASR	Risky	Benign	FPR	Seal
ambient_memory	33.33%	22.22%	100.00%	0.00%	0.00%
keyword_filter	33.33%	22.22%	100.00%	0.00%	0.00%
output_moderation	33.33%	22.22%	100.00%	0.00%	0.00%
provenance_only	24.38%	16.25%	96.25%	0.00%	0.00%
counterfactual_memory	3.96%	2.64%	91.25%	8.75%	0.00%
intent_capsules	0.00%	0.00%	100.00%	0.00%	8.33%

Intent capsules are the only tested defense in this split to jointly achieve 0.00% ASR, 0.00% risky action, 100.00% benign accuracy, and 0.00% FPR.

B. Poison Influence and Lifecycle Gap

Table V. Lifecycle-gap result.

Agent	ASR	Risky	Influence	Benign
output_moderation	34.05%	23.83%	34.05%	100.00%
semantic_output_judge	13.10%	9.17%	34.05%	100.00%
ablation_no_denied_actions	0.00%	0.00%	57.14%	100.00%
intent_capsules	0.00%	0.00%	0.00%	100.00%

C. Converted AgentDojo/InjecAgent Traces

Table VI. Converted external trace-corpus result.

Corpus	Out ASR	Out infl.	Cap ASR	Cap infl.	Cap seal
AgentDojo all	0.00%	80.65%	0.00%	0.00%	83.87%
InjecAgent all	0.00%	61.29%	0.00%	0.00%	61.48%
InjecAgent DH	0.00%	30.00%	0.00%	0.00%	30.39%
InjecAgent DS	0.00%	90.62%	0.00%	0.00%	90.62%

On converted AgentDojo and InjecAgent traces, output moderation and semantic judges can reach 0.00% final ASR but still allow 30.00%-90.62% poison influence. Intent capsules reach 0.00% ASR and 0.00% poison influence across all converted splits.

D. Stress and Live LLM Results

Table VII. Stress-suite examples.

Suite	Ambient ASR	Provenance ASR	Capsule ASR	Capsule benign
Generated holdout	44.37%	13.13%	0.00%	100.00%
Advanced suite	17.41%	2.59%	0.00%	100.00%

Table VIII. Medium live LLM planner result.

Condition	Rows	Tempted	Final ASR	Risky	Raw parse
ambient	108	22.22%	22.22%	22.22%	0.00%
capsule-filtered	108	2.78%	0.00%	0.00%	0.00%

Table IX. Defended live LLM result by model.

Model	Rows	Tempted	Final ASR	Risky	Raw parse
llama3	36	2.78%	0.00%	0.00%	0.00%
mistral	36	2.78%	0.00%	0.00%	0.00%
phi3	36	2.78%	0.00%	0.00%	0.00%

VIII. Discussion

The evaluation suggests that ASR alone is insufficient for memory-security evaluation. Output moderation may prevent a visible unsafe action while leaving poisoned memory influence intact. Intent-bound authorization instead rejects unauthorized influence before the memory becomes planning authority.

IX. Limitations

- The highest-volume benchmark uses a deterministic planner, although live LLM results are included as a realism check.
- Several corpora are generated, converted, or synthetic rather than collected from deployed user workflows.
- The raw-image multimodal/OCR pipeline is not fully evaluated.
- The prototype assumes policy code, verified identities, and cryptographic attestations are not compromised.

- The results support a defense layer under the stated threat model; they do not prove universal memory-poisoning security.

X. Conclusion

Persistent memory poisoning exploits the gap between retrieval and authority. Intent-bound memory capsules close this gap by requiring memories to carry explicit influence contracts. CapsuleGuard's results show that memory authorization can reduce both final attack success and poisoned planning influence while preserving benign utility in the tested prototype.

References

- [1] M. A. Ferrag, A. Lakas, N. Tihanyi, and M. Debbah, Securing LLM agents: From prompt sanitization to autonomous red teaming and beyond, Internet of Things and Cyber-Physical Systems, 2025.
- [2] J. Qian, Visual Inception: Compromising Long-term Planning in Agentic Recommenders via Multimodal Memory Poisoning, arXiv, 2026.
- [3] X. Yang, Y. He, S. Ji, B. Hooi, and J. S. Dong, Zombie Agents: Persistent Control of Self-Evolving LLM Agents via Self-Reinforcing Injections, arXiv, 2026.
- [4] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases, arXiv, 2024.
- [5] A. S. Patlan, A. Hebbar, P. Viswanath, and P. Mittal, Context manipulation attacks: Web agents are susceptible to corrupted memory, arXiv, 2025.
- [6] Z. Xu, X. Zhu, Y. Yao, M. Xue, and Y. Song, From Storage to Steering: Memory Control Flow Attacks on LLM Agents, arXiv, 2026.
- [7] H. Tian, Z. Sha, J. Wang, Y. Liu, Z. Huang, and X. Huang, InjecMEM: Memory Injection Attack on LLM Agent Memory Systems, OpenReview, 2025.
- [8] S. S. Srivastava and H. He, MemoryGraft: Persistent Compromise of LLM Agents via Poisoned Experience Retrieval, arXiv, 2025.
- [9] S. Dong et al., Memory Injection Attacks on LLM Agents via Query-Only Interaction, arXiv, 2025.
- [10] OWASP Foundation, OWASP Top 10 for Large Language Model Applications, 2025.
- [11] A. Jain and S. Suryawanshi, Sealing-Jutsu: CapsuleGuard prototype, <https://github.com/Mr-Akuma/Sealing-Jutsu>.